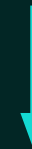


Selecting, Filtering, Sorting, and Creating Columns

Python 101 • Day 6 • DataFrame Operations

Today's move

from inspecting
a table



to asking
questions of it

select • filter • sort •
derive

BLOCK 42

What this block covers

30 minutes instruction + 30 minutes guided practice

Select

Choose one column or a smaller working table.

Filter

Keep rows using Boolean conditions.

Sort

Put records in a useful order.

Derive

Create new columns and quality labels.

Guiding question: What table operation am I trying to perform?

BLOCK 42

From inspection to action

Block 41 gave us a way to understand the table before changing it.



Do not skip the check step. Many pandas mistakes look plausible until you inspect the result.

BLOCK 42

Selecting one column

One column selection produces a Series.

```
statuses = records["status"]  
print(statuses.head())
```

**DataFrame $\approx$ table. Series $\approx$ one
labeled column.**

Mental model

index	status
0	active
1	active
2	inactive
3	active

Whole table $\rightarrow$ choose one named column
 $\rightarrow$ labeled column

Selecting several columns

Multiple-column selection returns a DataFrame.

```
summary = records[
    [
        "record_id",
        "status",
        "error_count",
    ]
]
```

Nested brackets

```
records[ list_of_column_names ]
```

The inner list names the columns to keep.
The outer brackets apply the selection to the dataframe.

One column: Series. Several columns: DataFrame.

Why column selection matters

Real tables often have many fields; analysis usually needs only a subset.

Raw dataset

50 columns

IDs
status
comments
file paths
metadata
extra fields



Working table

6 useful columns

record_id
source
status
completion
error_count
processing_status

Result

Less noise
Clearer code
Focused output

Filtering rows

Filtering keeps rows where a condition is True.

```
active_records = records[
    records["status"] == "active"
]
```

Step 1

Create a Boolean mask by comparing the status column to "active".

Step 2

Use `records[mask]` to keep only the rows where the mask value is True.

row	status	mask
0	active	True
1	inactive	False
2	active	True

Boolean mask mental model

This is the dataframe version of an if statement inside a loop.

Earlier loop pattern

```
active_records = []  
  
for record in records:  
    if record["status"] == "active":  
        active_records.append(record)
```

Condition evaluated one record at a time.



Dataframe pattern

```
active_records = records[  
    records["status"] == "active"  
]
```

Condition evaluated across the whole column.

BLOCK 42

Column comparisons

The familiar comparison operators work against columns.

==

equal to

!=

not equal to

>

greater than

>=

greater than or equal

<

less than

<=

less than or equal

```
high_error_records = records[
    records["error_count"] > 5
]

incomplete_records = records[
    records["completion"] < 100
]
```

Combining filter conditions

Use & and | for element-wise dataframe conditions.

AND-like

```
active_incomplete = records[
    (records["status"] == "active")
    & (records["completion"] < 100)
]
```

Both conditions must be True.

OR-like

```
needs_review = records[
    (records["error_count"] > 5)
    | (records["completion"] < 100)
]
```

Either condition can be True.

Beginner rule: put parentheses around each comparison before combining them.

Filtering with `isin()`

Use membership testing when several values are allowed.

```
allowed_sources = [  
    "maintenance",  
    "logistics",  
    "finance",  
]  
  
allowed_records = records[  
    records["source"].isin(  
        allowed_sources  
    )  
]
```

Connect to Day 3

Instead of asking one row at a time:

"Is this source in the allowed collection?"

pandas asks that question for the entire column.

`isin()` is easier to read than chaining many OR conditions.

Sorting records

Sorting changes row order so important records rise to the top.

Single column

```
sorted_records = records.sort_values(  
    "error_count",  
    ascending=False  
)
```

Largest error counts first.

Multiple columns

```
sorted_records = records.sort_values(  
    ["source", "error_count"],  
    ascending=[True, False]  
)
```

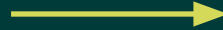
Group by source, then show biggest problems first.

Renaming columns

Column names are part of the dataset schema. Normalize them early.

Awkward source names

Record ID
Source System
Error Count



Consistent code names

```
records = records.rename(  
    columns={  
        "Record ID": "record_id",  
        "Source System": "source",  
        "Error Count": "error_count",  
    }  
)
```

Schema normalization is like value normalization: make later code predictable.

Creating calculated columns

Column arithmetic calculates a result for every row.

```
errors = records["error_count"]
count = records["record_count"]
complete = records["completion"]

records["error_rate"] = errors / count
records["remaining"] = 100 - complete
```

What happened?

No explicit for loop.

Pandas aligns the column values row by row and creates a new column in the table.

id	errors	count	error_rate	remaining
EQ-1	0	100	0.00	0
EQ-2	2	50	0.04	5

Creating a simple status column

.loc updates selected rows and selected columns.

```
df = records

df["quality_status"] = "ready"
err = df["error_count"]

df.loc[err > 0, "quality_status"] = "warning"
df.loc[err > 5, "quality_status"] = "rejected"
```

Read .loc as

For rows matching this
condition,
set this column
to this value.

**Order matters: rejected overwrites
warning when error_count > 5.**

When to use `.copy()`

Use `.copy()` when a filtered result becomes its own working dataset.

Risky intention

```
active_records = records[
    records["status"] == "active"
]
active_records["reviewed"] = True
```

The intention to modify a separate dataset is not clear.

Clear intention

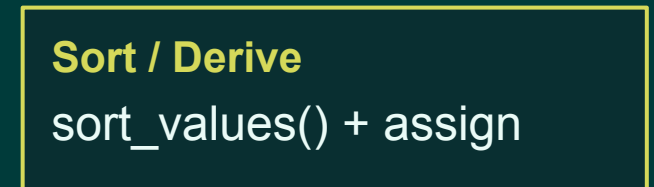
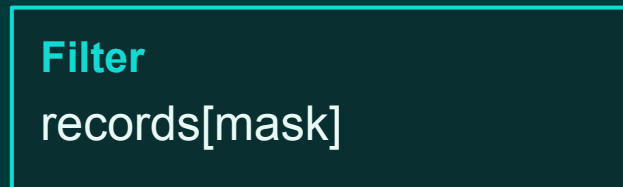
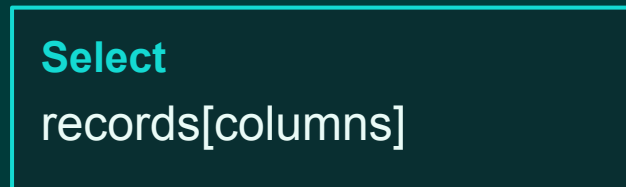
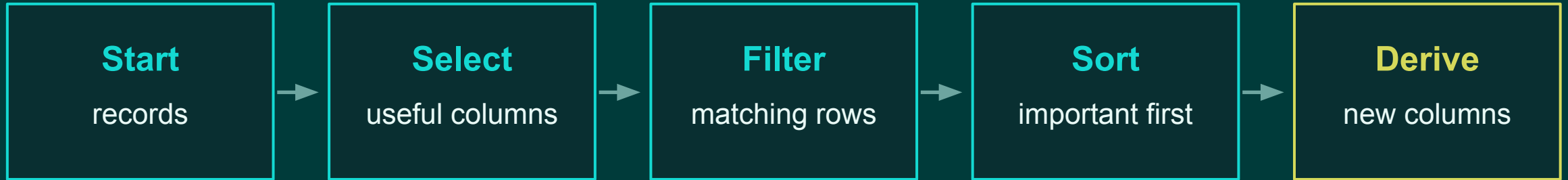
```
active_records = records[
    records["status"] == "active"
].copy()
active_records["reviewed"] = True
```

This says: treat `active_records` as its own table.

Classroom rule: filter + modify later → add `.copy()`.

A table operation workflow

Most dataframe transformations can be read as a small pipeline.



At each stage, inspect the shape or sample rows before moving on.

BLOCK 42

Guided practice setup

Use the dataframe loaded and inspected in Block 41.

Starting file

```
import pandas as pd

records = pd.read_csv(
    "data/raw/equipment_activity.csv"
)
```

Practice rule

Use dataframe operations.

Do not write an explicit for loop for these exercises.

Inspect each result with head() or shape.

Total practice time: 30 minutes

Exercise 1: select a working table

Goal: create a smaller dataframe with only useful fields.

```
working_records = records[
    [
        "record_id",
        "source",
        "status",
        "completion",
        "error_count",
    ]
].copy()

print(working_records.head())
```

Task

Run the selection, then compare `records.shape` and `working_records.shape`.

Timebox: 5 minutes

Exercise 2: filter active records

Goal: use a Boolean mask and compare before/after row counts.

```
active_records = working_records[
    working_records["status"] == "active"
]

print(working_records.shape)
print(active_records.shape)
```

Answer

How many records were removed by the filter?

What does that tell you about the dataset?

Timebox: 5 minutes

Exercises 3–4: multiple conditions and sorting

Goal: find active incomplete records, then sort problems first.

Exercise 3

```
active_incomplete = working_records[
    (working_records["status"] ==
     "active")
    & (working_records["completion"] <
     100)
]
```

Inspect with shape and head().

Exercise 4

```
problems_first =
active_incomplete.sort_values(
    "error_count",
    ascending=False
)

print(problems_first.head(10))
```

Show the ten most urgent rows.

Timebox: 8 minutes

Exercises 5–6: calculated and status columns

Goal: create new information from existing columns.

Exercise 5: remaining percent

```
df = working_records
completion = df["completion"]

df["remaining"] = (
    100 - completion
)
```

Then display only incomplete rows.

Exercise 6: quality status

```
df = working_records
err = df["error_count"]

df["quality_status"] = "ready"

df.loc[err > 0, "quality_status"] =
    "warning"
df.loc[err > 5, "quality_status"] =
    "rejected"
```

Timebox: 9 minutes

Applied challenge: answer table questions

Use dataframe operations only; no explicit for loop.

Questions 1–3

1. Which records have more than 5 errors?
2. Which active records are incomplete?
3. Which records come from maintenance or logistics?

Questions 4–6

4. Which five records have the highest error counts?
5. How many percentage points remain for each incomplete record?
6. Which records classify as ready, warning, or rejected?

Success: each answer is produced by a dataframe selection, filter, sort, or calculated column.

BLOCK 42

Block 42 wrap-up

You can now ask useful questions of a table.

Select

Keep the columns needed for the analysis.

Filter

Use masks to keep matching rows.

Sort

Put important records first.

Most pandas code should read like an operation on a table, not like a manual row-by-row script.

Next block: missing values, duplicates, data types, and cleaning decisions.